المدرسة الوطنية للعلوم التطبيقية - القنيطرة
National School of Applied Sciences - Kenitra

2ème Année Cycle Ingénieur

RAPPORT DE STAGE PFA

Titre du projet :

EMADS

vers une Data Science explicable par une architecture multi-agents hybride

Conception d'un pipeline statistiquement fondé, entre automatisation déterministe et arbitrage conditionnel par LLM

Réalisé par : Idriss Aaidoun
Niveau d'étude : 4ème année, filière Informatique, spécialité Big Data & IA
Établissement : École Nationale des Sciences Appliquées (ENSA) de Kénitra
Organisme d'accueil : Harmony Technology
Lieu de stage : Rabat, Maroc
Période de stage : 18 juin 2026 – 18 août 2026
Encadrant Académique : Monsieur Youssef Gahi
Responsable de stage : Madame Abir Bala, R&D Engineer – AI/Cybersecurity, Harmony Technology

Août 2026 – Rabat, Maroc

Remerciements

Je tiens à exprimer ma sincère gratitude à **Harmony Technology** pour m'avoir accueilli au sein de son équipe R&D IA/Cybersécurité et m'avoir donné l'opportunité de contribuer, durant ces deux mois, à un projet de recherche exigeant et stimulant.

Mes remerciements les plus sincères s'adressent à **Madame Abir Bala**, ma responsable de stage, Ingénieure R&D en Intelligence Artificielle et Cybersécurité, pour la qualité de son encadrement, la clarté de ses orientations scientifiques et sa disponibilité constante tout au long de ce stage. Sa vision du projet – fonder l'explicabilité d'un système multi-agents sur des méthodes statistiques rigoureuses plutôt que sur des justifications approximatives générées par un LLM – a structuré l'ensemble de ma démarche et constitue le fil conducteur de ce rapport.

J'adresse également mes remerciements à **Monsieur Youssef Gahi**, mon encadrant académique à l'ENSA de Kénitra, pour le suivi apporté à ce stage et pour ses conseils sur la structuration de ce travail.

Enfin, je remercie l'ENSA de Kénitra et l'ensemble du corps enseignant de la filière Informatique, spécialité Big Data & IA, pour la formation qui m'a permis d'aborder ce stage avec les compétences techniques et scientifiques nécessaires.

Résumé

Ce rapport présente EMADS (*Explainable Multi-Agent Data Science System*), un système multi-agents conçu durant un stage de fin d'année chez Harmony Technology, dont l'objectif dépasse la simple automatisation d'un pipeline de *Machine Learning*. Là où les solutions d'AutoML classiques traitent la sélection de modèle et l'optimisation comme des *black boxes*, ce travail interroge une question de recherche plus large : comment un agent artificiel prend-il ses décisions, et comment rendre ces décisions explicables et fiables ?

La contribution centrale consiste à **fonder statistiquement l'explicabilité du système à chaque étape de décision**, plutôt que de la déléguer à des justifications génériques produites par un grand modèle de langage (*Large Language Model*, LLM). Cette exigence a conduit à une architecture hybride workflow/agentique : EMADS orchestre neuf agents séquentiels au sein d'un pipeline structurellement déterministe, et ne bascule en comportement agentique qu'à trois points précis, lorsqu'un test statistique rigoureux — test de Wilcoxon signed-rank, corrélation de rang de Spearman, hiérarchie de confiance graduée — s'avère lui-même inconclusif. Chaque arbitrage est tracé et justifié, garantissant que l'explicabilité du système repose sur des statistiques vérifiables plutôt que sur des rationalisations générées *a posteriori*.

La validation expérimentale, menée sur quatre jeux de données (*datasets*) délibérément conçus pour être difficiles, a permis d'identifier et de corriger neuf anomalies réelles, renforçant la fiabilité globale du système. Elle a par ailleurs mis au jour une limite méthodologique majeure du choix initial de validation croisée : avec seulement cinq plis (*folds*), le test de Wilcoxon signed-rank possède un plancher de p-value structurel ($p_{min} = 0{,}0625$) le rendant statistiquement incapable de conclure, quelle que soit l'ampleur de l'écart de performance observé. La correction de ce défaut constitue un apport méthodologique propre à ce travail, au-delà de la simple implémentation logicielle.

Mots-clés : systèmes multi-agents, intelligence artificielle explicable (XAI), SHAP, AutoML, validation statistique de l'explicabilité, LangGraph, apprentissage automatique.

Table des matières

Remerciements **1**

Résumé **2**

Introduction générale **5**

1 Contexte du stage **7**
1.1 Présentation de l'organisme d'accueil 7
1.1.1 Harmony Technology 7
1.1.2 L'équipe Recherche & Développement IA/Cybersécurité 7
1.2 Contexte et problématique du stage 8
1.2.1 Le besoin d'explicabilité en science des données 8
1.2.2 Cahier des charges 8
1.3 Méthodologie de travail 9

2 État de l'art **10**
2.1 Agents et workflows : un cadrage théorique nécessaire 10
2.2 Frameworks d'agents pour la Data Science 10
2.3 Explicabilité en intelligence artificielle (XAI) 11
2.3.1 Panorama et enjeux 11
2.3.2 SHAP : fondements et garanties théoriques 11
2.3.3 Explicabilité et fiabilité des agents IA 12
2.4 Benchmarks et évaluation des systèmes d'AutoML 12
2.5 Synthèse 12

3 Analyse des besoins **14**
3.1 Besoins fonctionnels 14
3.2 Besoins non fonctionnels 14
3.3 Cas d'utilisation 15
3.4 Choix technologiques 15

4 Architecture **16**
4.1 Vue d'ensemble 16
4.2 Vue utilisateur : diagramme de cas d'utilisation 17
4.3 Structure du code : diagramme de classes 18

4.4 Le pipeline : vue d'ensemble des neuf agents 19
4.5 L'architecture hybride workflow/agentique 19
4.6 Décisions de conception rejetées 21
4.7 Structure des dossiers du projet 21

5 Implémentation **22**
5.1 Vue d'ensemble 22
5.2 Agents à logique déterministe 22
5.3 Model Selection Agent : arbitrage statistique 23
5.4 Explainability Agent : accord SHAP / Permutation Importance 24
5.5 Data Understanding Agent et Hyperparameter Optimization Agent 25
5.6 Traçabilité : `AgentDecision` et `EMADSState` 25
5.7 Justification du choix de CV_FOLDS = 10 26
5.8 Défis techniques d'implémentation 27

6 Validation et résultats **28**
6.1 Méthodologie de validation 28
6.2 Neuf anomalies détectées et corrigées 29
6.3 Preuve empirique du plancher de p-value de Wilcoxon 30
6.4 Résultats d'explicabilité et d'évaluation 30
6.5 Traçabilité en pratique 31
6.6 Aperçu de l'interface utilisateur 31
6.7 Synthèse 32

7 Discussion, limites et perspectives **33**
7.1 Une question de conception au-delà du cahier des charges 33
7.2 Réponses aux remarques formulées en réunion de suivi 34
7.3 Limites 34
7.4 Perspectives d'évolution 35

Conclusion générale **36**

Bibliographie **37**

A Captures d'écran complémentaires de l'interface **38**

B Exemple de journal de traçabilité **39**

C Définition complète de `EMADSState` **40**

Introduction générale

L'intelligence artificielle occupe une place croissante dans les processus de décision, y compris dans des domaines où ces décisions doivent pouvoir être vérifiées, justifiées et remises en question. Cette exigence est particulièrement forte en science des données : construire un modèle de *Machine Learning* performant ne suffit plus si les choix ayant mené à ce modèle – sélection des variables, choix de l'algorithme, réglage des hyperparamètres – restent opaques. Les solutions d'AutoML existantes ont largement résolu le problème de l'automatisation, mais traitent le plus souvent ces choix comme des boîtes noires, sans offrir de justification vérifiable de leurs décisions.

C'est dans ce contexte que s'inscrit ce stage de fin d'année, réalisé au sein de l'équipe Recherche & Développement en Intelligence Artificielle et Cybersécurité de Harmony Technology, sous la responsabilité de Madame Abir Bala. La mission confiée dépasse la construction d'un pipeline d'automatisation supplémentaire : il s'agit de concevoir un *Data Science Agent* capable non seulement de mener à bien les étapes d'un projet de *Machine Learning* – analyse exploratoire, prétraitement, sélection de modèle, optimisation, évaluation – mais aussi d'expliquer, pour chacune de ses décisions, pourquoi elle a été prise, avec quel niveau de confiance, et selon quels critères.

Cette exigence soulève une question de recherche à part entière, qui structure l'ensemble de ce rapport : **comment un système multi-agents peut-il automatiser un pipeline de science des données tout en rendant ses décisions explicables et dignes de confiance, plutôt que de se contenter de produire, après coup, une justification en langage naturel ?** Répondre par un simple appel à un grand modèle de langage (LLM) chargé de "raconter" une explication plausible aurait été insuffisant : une telle explication ne serait ni vérifiable, ni fiable, ni reproductible.

Le système conçu durant ce stage, EMADS (*Explainable Multi-Agent Data Science System*), répond à cette question par une architecture hybride entre automatisation déterministe et arbitrage conditionnel. EMADS est structurellement un *workflow* : une séquence fixe de neuf agents spécialisés traite chaque jeu de données de bout en bout, de la compréhension des données jusqu'à la génération d'un rapport final. Ce pipeline devient cependant agentique en trois points précis de décision, où un test statistique rigoureux – et lui seul – détermine si la situation est suffisamment claire pour être tranchée automatiquement, ou si elle relève d'une zone d'incertitude nécessitant un arbitrage supplémentaire. Ce n'est que dans ce second cas que le LLM intervient, selon des critères secondaires explicites et jamais pour recalculer un score déjà déterminé statistiquement.

Ce rapport se propose de documenter cette démarche dans son ensemble. Le premier chapitre présente le contexte du stage et l'entreprise d'accueil. Le deuxième situe EMADS par rapport à l'état de l'art des systèmes multi-agents, de l'AutoML et de l'IA explicable (XAI). Le troisième détaille l'analyse des besoins et les choix technologiques retenus. Le quatrième chapitre expose la conception du système et développe la contribution centrale de ce travail : l'architecture hybride workflow/agentique. Le cinquième chapitre décrit l'implémentation concrète de chacun des neuf agents. Le sixième présente la méthodologie de validation expérimentale et ses résultats, dont une découverte méthodologique portant sur une limite structurelle du test statistique initialement retenu. Enfin, le septième chapitre discute les limites du système, avant qu'une conclusion générale ne revienne sur les apports de ce travail et trace les perspectives d'évolution du projet.

Chapitre 1

Contexte du stage

1.1 Présentation de l'organisme d'accueil

1.1.1 Harmony Technology

Ce stage de fin d'année s'est déroulé au sein de **Harmony Technology**, société marocaine spécialisée dans la conception et le développement de solutions technologiques pour les institutions, les grandes entreprises et les industriels. L'entreprise agit également comme fournisseur et intégrateur de solutions dans les domaines des NTIC, de l'IoT (*Internet of Things*) et de la RFID. Son portefeuille de clients couvre un spectre large d'organisations exigeantes en matière de fiabilité technologique : instances gouvernementales, bibliothèques, centres de recherche, établissements d'enseignement académique, administrations publiques et écoles scientifiques.

Le siège social de Harmony Technology est situé Villa N°14, Rue Annassime, Bloc M, Secteur 9, Hay Riad, à Rabat. Le stage s'est déroulé au sein du bureau de l'équipe R&D, situé au 2 à 9 Rue Al Karam, également à Rabat. Au-delà du Maroc, Harmony Technology dispose d'une présence internationale étendue, avec des implantations en Afrique (Rwanda, Côte d'Ivoire, Gabon, Congo, Sénégal), au Moyen-Orient (Oman, Égypte, Émirats arabes unis) et en Europe (Italie, Royaume-Uni, Suisse) – une empreinte géographique qui reflète la vocation de l'entreprise à accompagner la transformation numérique de ses clients au-delà du seul marché marocain.

1.1.2 L'équipe Recherche & Développement IA/Cybersécurité

Le stage a été réalisé au sein de l'équipe R&D dédiée à l'Intelligence Artificielle et à la Cybersécurité, sous la responsabilité de **Madame Abir Bala**, Ingénieure R&D. Cette équipe se distingue des équipes projet classiques de l'entreprise par sa vocation exploratoire : plutôt que de livrer des solutions directement destinées à un client, elle mène des travaux de recherche appliquée susceptibles, à terme, d'alimenter l'offre technologique de Harmony Technology ou de faire l'objet d'une valorisation scientifique. C'est dans ce cadre que s'inscrit la mission confiée pour ce stage, présentée en détail dans la section suivante.

1.2 Contexte et problématique du stage

1.2.1 Le besoin d'explicabilité en science des données

La mission confiée pour ce stage a été formulée par Madame Abir Bala autour d'un constat simple mais structurant : automatiser un pipeline de *Machine Learning* est aujourd'hui un problème largement résolu par les solutions d'AutoML existantes, mais ces solutions traitent le plus souvent les décisions prises en cours de route – quel modèle retenir, selon quels critères, avec quel degré de certitude – comme des boîtes noires. Or, dans un contexte où les résultats d'un système d'IA doivent pouvoir être audités, remis en question ou justifiés auprès d'un tiers, cette opacité constitue une limite critique.

La mission ne consistait donc pas à construire un pipeline d'automatisation supplémentaire, mais à concevoir un ***Data Science Agent*** : un système autonome capable de mener à bien les étapes principales d'un projet de *Machine Learning* – analyse exploratoire des données (EDA), prétraitement et nettoyage, sélection ou création de variables, choix du modèle, optimisation des hyperparamètres, évaluation des performances – tout en produisant un rapport documentant explicitement les choix effectués. La véritable question de recherche posée par l'encadrante portait sur le *comment* : comment un agent prend-il ses décisions, et comment rendre ces décisions explicables et fiables ? À titre d'illustration, lorsque le système sélectionne un modèle de *Machine Learning* plutôt qu'un autre, il doit être en mesure d'expliquer pourquoi ce modèle a été retenu, quel niveau de confiance il associe à ce choix, et quels critères ont influencé sa décision.

1.2.2 Cahier des charges

La première phase de la mission, formulée explicitement par l'encadrante, consistait à réaliser un état de l'art couvrant quatre axes :

— les agents IA appliqués à la science des données et à l'AutoML ;
— les *frameworks* d'agents existants (LangGraph, CrewAI, AutoGen) ;
— les travaux récents sur l'explicabilité et l'évaluation des agents IA ;
— les *benchmarks* et jeux de données utilisés pour évaluer les systèmes d'AutoML.

Cet état de l'art, présenté au Chapitre 2, a servi de socle théorique à la conception du système. L'objectif final fixé pour le stage était double : aboutir à un **prototype fonctionnel**, et produire une **évaluation scientifique** suffisamment rigoureuse pour envisager, à terme, une valorisation sous forme de publication.

1.3 Méthodologie de travail

Le stage s'est déroulé selon une démarche itérative, jalonnée de points d'avancement réguliers avec l'encadrante. Après la phase initiale de recherche bibliographique, la conception du système s'est faite par construction progressive du pipeline agent par agent, chaque agent étant testé individuellement avant son intégration dans le *pipeline* complet orchestré par le Supervisor Agent. Cette approche progressive a permis d'identifier et de corriger, au fil de l'eau, plusieurs anomalies de conception qui n'auraient pu être détectées qu'à l'échelle du système complet – une méthodologie de validation détaillée au Chapitre 6, où sont notamment décrits les quatre jeux de données volontairement conçus pour éprouver la robustesse du système sur des cas limites.

Un point d'étape formel avec l'encadrante, portant sur l'état d'avancement du prototype et la solidité de son explicabilité statistique, a permis de valider l'orientation générale du projet avant d'engager la phase de rédaction du présent rapport.

Chapitre 2

État de l'art

2.1 Agents et workflows : un cadrage théorique nécessaire

Le terme "agent" recouvre des réalités très différentes selon le degré d'autonomie accordé au système. Selon la définition classique de Russell et Norvig, un agent est une entité qui perçoit son environnement et agit sur lui pour atteindre un objectif. Cette définition n'implique cependant rien sur la manière dont l'agent choisit ses actions. Une distinction plus opérationnelle, proposée par Anthropic dans *Building Effective Agents*, sépare les systèmes en deux catégories : un **workflow** suit un chemin d'exécution prédéfini par le développeur, même lorsque ce chemin comporte des branchements conditionnels ; un **agent**, au sens strict, dirige lui-même dynamiquement son propre déroulement.

Cette distinction structure directement la conception d'EMADS, développée au Chapitre 4 : le système y est présenté comme un workflow qui devient ponctuellement agentique, plutôt que comme un agent autonome de bout en bout. Ce choix n'est pas neutre : un système pleinement agentique, où un LLM orchestrerait librement l'enchaînement des étapes, sacrifierait la reproductibilité nécessaire à toute validation statistique des résultats – un compromis jugé inacceptable au regard de l'objectif d'explicabilité fixé pour ce stage.

2.2 Frameworks d'agents pour la Data Science

Trois *frameworks* d'orchestration d'agents dominent aujourd'hui l'écosystème Python : LangGraph, CrewAI et AutoGen. Les trois permettent de faire collaborer plusieurs agents autour d'un objectif commun, mais diffèrent sur un point déterminant pour ce projet : le degré de contrôle laissé au développeur sur le flux d'exécution. CrewAI et AutoGen sont conçus autour d'une logique de délégation entre agents relativement libre, proche d'un fonctionnement pleinement agentique. LangGraph, à l'inverse, modélise explicitement le pipeline comme un graphe d'états (*StateGraph*) : chaque transition entre agents est définie explicitement, ce qui permet d'imbriquer un comportement conditionnel ponctuel (l'arbitrage LLM) dans une structure par ailleurs entièrement déterministe. C'est ce contrôle

fin sur le flux d'exécution, indispensable à la démarche de validation statistique retenue pour EMADS, qui a motivé le choix de LangGraph pour ce projet.

2.3 Explicabilité en intelligence artificielle (XAI)

2.3.1 Panorama et enjeux

L'explicabilité en intelligence artificielle (*eXplainable AI*, XAI) désigne l'ensemble des méthodes permettant de rendre compréhensibles les décisions d'un système par ailleurs difficile à interpréter directement. Ali et al. (2023) proposent une taxonomie de ces méthodes selon quatre axes – portée (locale ou globale), moment d'application (avant ou après l'entraînement du modèle), spécificité (propre à un modèle ou agnostique), et type de sortie produite (numérique, visuelle, ou en langage naturel). Cette taxonomie éclaire un choix de conception central d'EMADS : le système combine une méthode d'explicabilité numérique et rigoureuse (SHAP) avec une restitution en langage naturel, mais sans jamais confondre les deux rôles – le calcul reste statistique, la formulation en langage naturel n'intervient qu'ensuite, pour restituer un résultat déjà établi.

Deux grandes familles de méthodes d'attribution dominent la littérature : les approximations locales, dont LIME (Ribeiro et al., 2016) est le représentant le plus connu, et les méthodes fondées sur la théorie des jeux coopératifs, dont SHAP est aujourd'hui la référence.

2.3.2 SHAP : fondements et garanties théoriques

SHAP (*SHapley Additive exPlanations*, Lundberg & Lee, 2017) repose sur la valeur de Shapley, issue de la théorie des jeux coopératifs (Shapley, 1953). Pour une prédiction $f(x)$ et un ensemble de features F, la contribution attribuée à une feature i est donnée par :

$$\varphi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!\,(|F| - |S| - 1)!}{|F|!} \left[f(S \cup \{i\}) - f(S) \right] \tag{2.1}$$

Cette formule garantit quatre axiomes que Shapley (1953) démontre uniques à cette méthode d'attribution : *Efficiency* (la somme des contributions égale la prédiction totale), *Symmetry*, *Dummy*, et *Additivity*. Aucune autre méthode d'attribution, y compris LIME, ne satisfait simultanément ces quatre propriétés – ce qui justifie le choix de SHAP comme fondement de l'explicabilité d'EMADS plutôt qu'une approximation locale plus légère mais moins garantie.

Le calcul exact de φ_i est de complexité exponentielle en nombre de features. TreeSHAP exploite la structure des modèles à base d'arbres (Random Forest, XGBoost, LightGBM)

pour ramener ce calcul à une complexité polynomiale, sans perte d'exactitude – ce qui explique pourquoi EMADS privilégie ces familles de modèles et recourt à KernelSHAP, une approximation, uniquement lorsque le modèle sélectionné n'est pas à base d'arbres.

2.3.3 Explicabilité et fiabilité des agents IA

Au-delà de l'explicabilité d'un modèle pris isolément, la littérature récente sur les agents IA pose une question plus large : comment évaluer la fiabilité d'un système dont les décisions résultent de plusieurs étapes enchaînées, potentiellement interdépendantes ? Dietterich (1998) a montré, dans le cadre de la comparaison d'algorithmes d'apprentissage, qu'une différence de performance observée entre deux modèles n'est significative que si elle est établie par un test statistique adapté – une simple comparaison de scores moyens, sans test de significativité, expose à des conclusions non reproductibles. Ce principe, rarement appliqué dans les systèmes d'agents IA existants où l'arbitrage repose le plus souvent sur le jugement qualitatif d'un LLM, constitue le socle méthodologique sur lequel s'appuie EMADS : chaque arbitrage y est conditionné à un test statistique explicite plutôt qu'à une évaluation qualitative.

Alvarez-Melis et Jaakkola (2018) soulignent par ailleurs un risque propre aux explications produites par des méthodes locales : leur instabilité, c'est-à-dire la possibilité que deux exécutions proches produisent des explications sensiblement différentes. Ce risque, documenté au Chapitre 6, motive en partie les perspectives d'évaluation de la stabilité temporelle des explications SHAP identifiées pour une version future du système.

2.4 Benchmarks et évaluation des systèmes d'AutoML

L'évaluation des systèmes d'AutoML pose une difficulté méthodologique récurrente dans la littérature : la plupart des *benchmarks* existants mesurent la performance brute des modèles produits (précision, score F1, RMSE), mais rarement la qualité ou la fiabilité des explications associées à ces choix. Cette lacune renforce la pertinence de la démarche retenue pour EMADS, dont la validation, présentée au Chapitre 6, porte volontairement sur des jeux de données construits pour éprouver spécifiquement la robustesse des mécanismes d'arbitrage et d'explicabilité, plutôt que sur la seule performance finale des modèles produits.

2.5 Synthèse

Cet état de l'art dégage trois éléments qui structurent directement la conception d'EMADS, développée au Chapitre 4 : la nécessité de distinguer clairement workflow et comportement agentique pour préserver la reproductibilité ; la supériorité théorique de

SHAP sur les méthodes d'attribution locales pour fonder une explicabilité vérifiable ; et l'absence, dans la littérature existante, d'une exigence systématique de validation statistique des décisions d'un agent – exigence qu'EMADS place au centre de sa conception.

Chapitre 3

Analyse des besoins

3.1 Besoins fonctionnels

Le système doit permettre, à partir d'un jeu de données fourni par l'utilisateur, de :

- réaliser une analyse exploratoire des données (statistiques descriptives, corrélations, détection d'outliers) ;
- effectuer un prétraitement adapté au contenu du dataset (imputation, encodage, mise à l'échelle) ;
- sélectionner un modèle de *Machine Learning* parmi plusieurs candidats, en justifiant ce choix ;
- optimiser les hyperparamètres du modèle retenu, sous contrainte de temps ;
- évaluer les performances du modèle avec un intervalle de confiance ;
- produire une explication statistique des décisions prises à chaque étape, et non une reformulation en langage naturel a posteriori ;
- générer un rapport PDF consolidant l'ensemble de ces éléments, et permettre le téléchargement du modèle entraîné (`.pkl`).

3.2 Besoins non fonctionnels

Trois exigences non fonctionnelles ont guidé les choix de conception, développés au Chapitre 4 :

- **Traçabilité** : chaque décision prise par un agent doit pouvoir être retracée à son déclencheur exact (statistique ou LLM), via le journal `llm_arbitration_log`.
- **Reproductibilité** : à jeu de données et graine aléatoire identiques, le système doit produire un résultat stable – une exigence qui exclut une orchestration pleinement agentique du pipeline (cf. section 2.1).
- **Robustesse** : le système doit rester fonctionnel face à des données imparfaites (valeurs manquantes, colonnes mal formées, classes déséquilibrées), sous un budget de temps borné plutôt qu'un temps de calcul illimité.

3.3 Cas d'utilisation

L'utilisateur du système – data scientist ou analyste métier – interagit avec EMADS selon un cas d'utilisation principal : soumettre un jeu de données et obtenir en retour un modèle entraîné accompagné de son rapport d'explicabilité. Ce cas d'utilisation se décompose en sous-cas indépendants : consulter le rapport d'analyse exploratoire, consulter le détail d'un arbitrage LLM tracé dans le journal, télécharger le modèle entraîné, et relancer une analyse sur un nouveau jeu de données. La représentation graphique de ces interactions est présentée au Chapitre 4, avec les autres schémas de conception du système.

3.4 Choix technologiques

Le tableau 3.1 synthétise les choix technologiques retenus et leur justification au regard des besoins précédents.

Composant	**Choix**	**Justification**
Orchestration	LangGraph	Graphe d'états explicite, seul cadre permettant d'imbriquer un arbitrage conditionnel dans un pipeline reproductible (cf. section 2.1)
Modèles ML	scikit-learn, XGBoost, LightGBM	Compatibles TreeSHAP (calcul exact, complexité polynomiale)
Optimisation	Optuna (TPE)	Recherche bayésienne efficace sous budget de temps borné
Explicabilité	SHAP	Seule méthode d'attribution garantissant les 4 axiomes de Shapley (cf. section 2.3.2)
LLM d'arbitrage	Groq (llama-3.1-8b-instant)	Faible latence, suffisant pour un rôle d'arbitrage ponctuel et non de génération de contenu analytique
Rapport	ReportLab	Génération PDF programmatique, contrôle fin de la mise en page
Interface	Streamlit	Prototypage rapide d'une UI interactive pour un usage data science

TABLE 3.1 – Choix technologiques et justification.

Chapitre 4

Architecture

4.1 Vue d'ensemble

EMADS est orchestré par un **Supervisor Agent** qui pilote l'exécution séquentielle de neuf agents spécialisés au sein d'un *StateGraph* LangGraph. L'ensemble des agents partage un état unique, `EMADSState` (`TypedDict`), qui transporte de bout en bout les données du dataset, les décisions prises et leurs niveaux de confiance associés (figure 4.1).

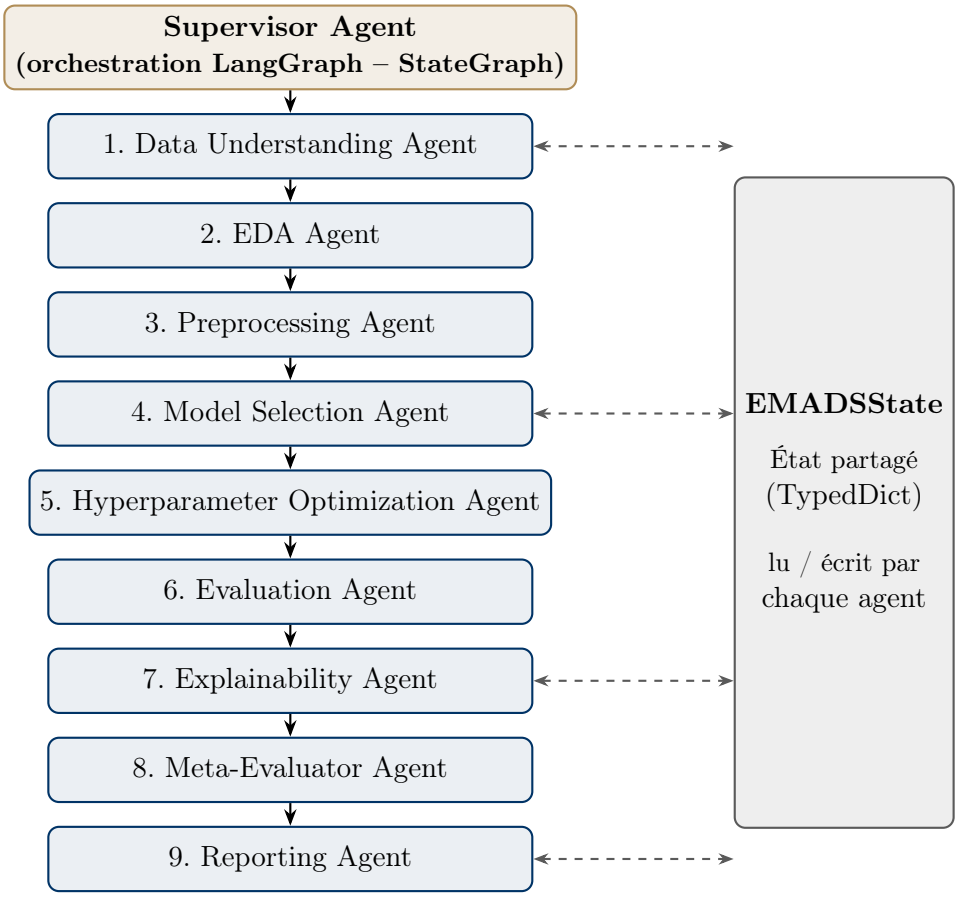


FIGURE 4.1 – Architecture globale d'EMADS : Supervisor Agent, pipeline séquentiel des neuf agents, état partagé `EMADSState`.

4.2 Vue utilisateur : diagramme de cas d'utilisation

Les interactions décrites au Chapitre 3 sont représentées figure 4.2 sous forme de diagramme de cas d'utilisation UML.

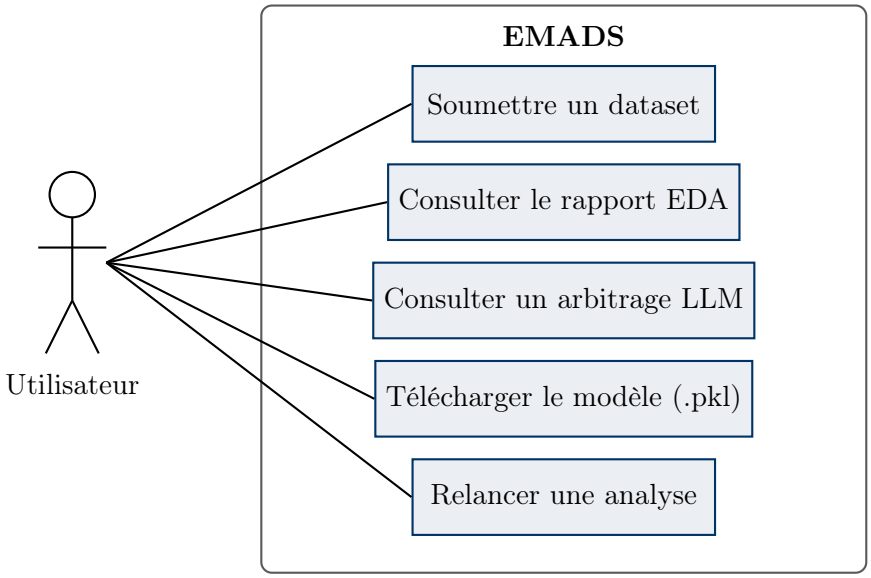


FIGURE 4.2 – Diagramme de cas d'utilisation d'EMADS.

4.3 Structure du code : diagramme de classes

Trois éléments structurent le code du pipeline : `BaseAgent`, classe abstraite dont héritent les neuf agents ; `AgentDecision`, qui encapsule pour chaque décision sa valeur, son niveau de confiance et son raisonnement ; et `EMADSState`, l'état partagé qui transite entre tous les agents (figure 4.3).

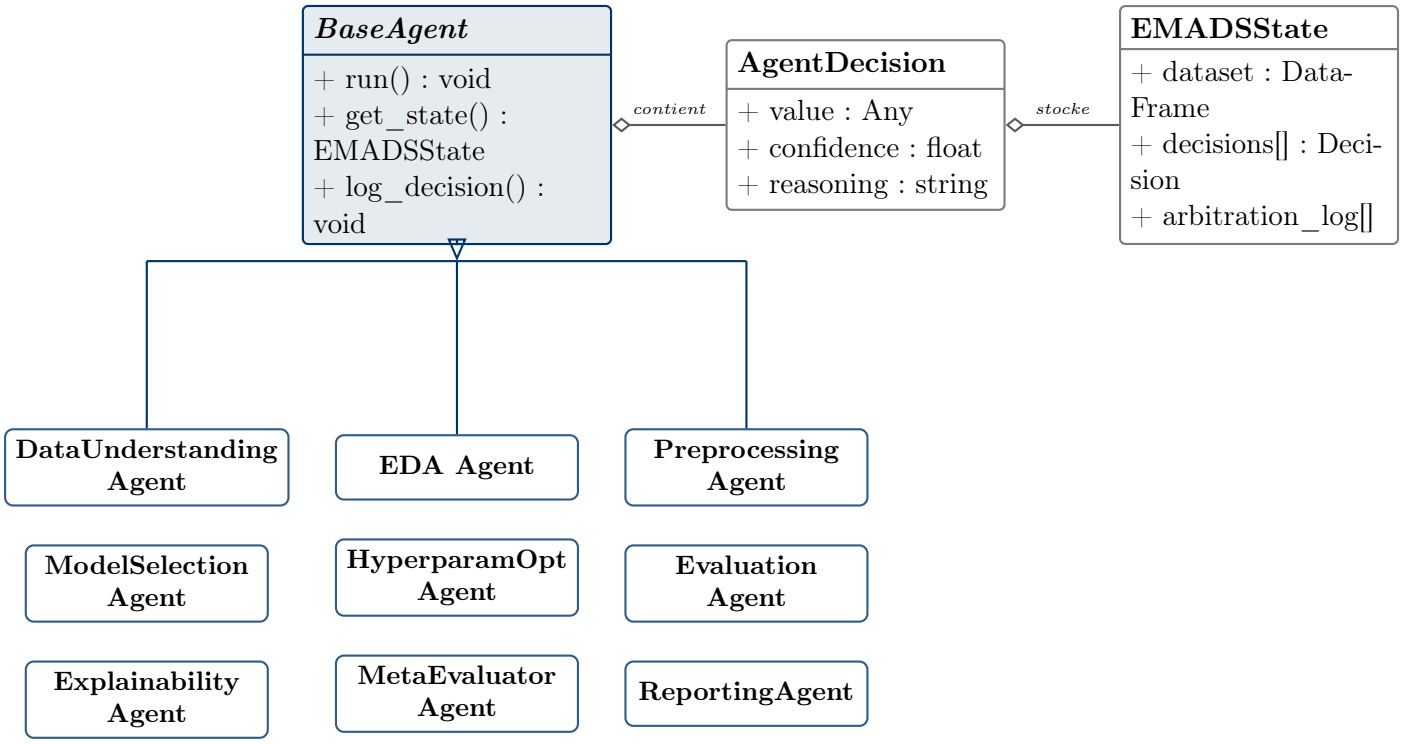


FIGURE 4.3 – Diagramme de classes simplifié : `BaseAgent`, les neuf agents spécialisés (relation d'héritage), `AgentDecision` et `EMADSState` (relations d'association).

4.4 Le pipeline : vue d'ensemble des neuf agents

Le Supervisor Agent enchaîne, dans un ordre fixe, les neuf agents présentés figure 4.1. Le détail de leur implémentation est présenté au Chapitre 5 ; cette section se concentre sur l'articulation d'ensemble et sur les trois points où cette séquence, par ailleurs déterministe, bascule en comportement agentique – objet de la section suivante, qui constitue la contribution centrale de ce travail.

4.5 L'architecture hybride workflow/agentique

La distinction théorique entre workflow et agent, posée au Chapitre 2, trouve ici sa traduction concrète. EMADS est structurellement un *workflow* : la séquence des neuf agents est fixe et ne varie jamais d'une exécution à l'autre – une condition nécessaire à l'objectif d'explicabilité statistique qui structure ce rapport (cf. Résumé), et non une limitation.

Le système devient agentique à **trois points de décision** précis, où un test statistique – et lui seul – détermine si la situation est suffisamment claire pour être tranchée automatiquement, ou si elle appelle un arbitrage par LLM. Dans les trois cas, le LLM n'intervient jamais pour recalculer un résultat déjà établi statistiquement.

Point 1 – Sélection de modèle

Un test de Wilcoxon signed-rank est appliqué sur les scores de validation croisée *par fold* entre le meilleur modèle et le second. Si $p > 0{,}05$, le LLM est sollicité pour arbitrer selon des critères secondaires (interprétabilité, coût de calcul), avec $confidence = 1 - p$. Ce comportement, illustré figure 4.4, est justifié par la règle du *one standard error* (Breiman et al., 1984).

Point 2 – Compréhension des données

La détection de la colonne cible repose sur une hiérarchie de confiance à quatre niveaux. L'arbitrage LLM se déclenche inconditionnellement dès qu'aucun nom n'est reconnu et qu'aucune cible n'est fournie.

Point 3 – Explicabilité

SHAP et Permutation Importance sont comparées via une corrélation de rang de Spearman. Si cette corrélation est inférieure à 0,3, le LLM explique la nature du désaccord sans jamais trancher en faveur de l'une des deux méthodes, avec $confidence = \max(0, \text{agreement_score})$.

Chaque arbitrage est tracé dans `llm_arbitration_log` avec son déclencheur statistique exact et la réponse du LLM. Le diagramme de séquence ci-dessous illustre ce mécanisme pour le point 1, le plus représentatif des trois.

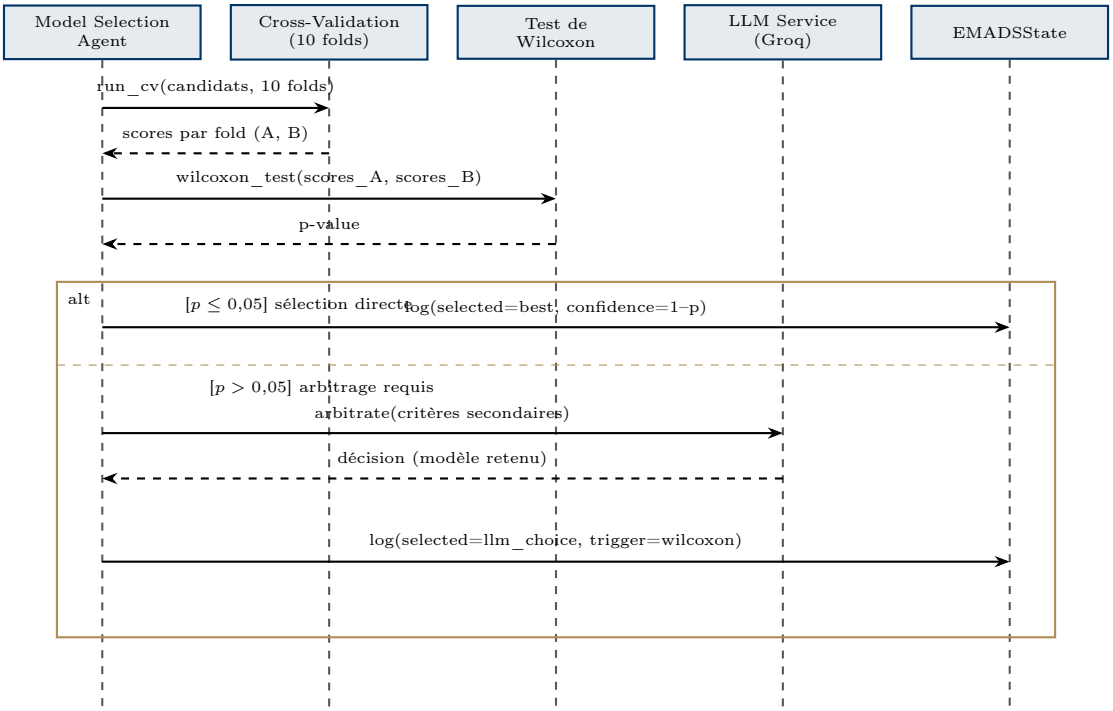


FIGURE 4.4 – Diagramme de séquence de l'arbitrage Model Selection : du test de Wilcoxon signed-rank à la décision finale, avec journalisation systématique dans `EMADSState`.

4.6 Décisions de conception rejetées

Trois choix d'architecture alternatifs ont été explicitement écartés :

— **Pas de dossier `tools/`** façon LangChain *function-calling* : EMADS est un pipeline séquentiel spécialisé, pas un agent à appel d'outils dynamique.

— **Pas de dossier `prompts/`** séparé : aucune duplication réelle ne le justifiait.

— **Pas d'architecture pleinement agentique** : ce choix aurait sacrifié la reproductibilité nécessaire à la validation statistique, exigence centrale de ce travail (cf. section 4.5).

4.7 Structure des dossiers du projet

```
emads/
|-- app/
|   |-- core/
|   |   |-- agents/                    # Les 9 agents du pipeline
|   |   |   |-- base_agent.py          # Contrat commun (BaseAgent)
|   |   |   |-- data_understanding_agent.py
|   |   |   |-- eda_agent.py
|   |   |   |-- preprocessing_agent.py
|   |   |   |-- model_selection_agent.py
|   |   |   |-- hyperparameter_agent.py
|   |   |   |-- evaluation_agent.py
|   |   |   |-- explainability_agent.py
|   |   |   |-- meta_evaluator_agent.py
|   |   |   `-- reporting_agent.py
|   |   |-- state/
|   |   |   `-- emads_state.py         # EMADSState + AgentDecision
|   |   `-- supervisor/
|   |       `-- supervisor_agent.py    # Graphe d'execution LangGraph
|   |-- services/
|   |   |-- llm_service.py             # Client Groq + fallback deterministe
|   |   `-- pdf_service.py             # Generation du rapport PDF
|   |-- ui/
|   |   |-- streamlit_app.py           # Dashboard Streamlit
|   |   `-- theme.py                   # Themes, badges et styles CSS
|   `-- utils/
|       |-- plot_utils.py
|       |-- file_utils.py
|       `-- logger.py
|-- data/
|   |-- uploads/                       # Datasets CSV utilisateurs
|   `-- outputs/                       # Artefacts generes
|       |-- clean_dataset.csv          # Donnees pretraitees
|       |-- models/selected_model.pkl  # Modele final entraine
|       `-- plots/                     # Visualisations EDA/SHAP (PNG)
|-- logs/                              # Logs d'execution
|-- reports/                           # Rapports PDF generes
|-- tests/                             # Suite de tests unitaires
|-- requirements.txt                   # Dependances Python
|-- run.py                             # Execution en CLI
`-- README.md
```

Chapitre 5

Implémentation

5.1 Vue d'ensemble

Ce chapitre détaille l'implémentation des trois mécanismes d'arbitrage présentés au Chapitre 4, ainsi que la structure de traçabilité qui les sous-tend. Le rôle des agents à logique purement déterministe est résumé section 5.2 ; les extraits ci-après sont issus tels quels du code source du projet.

5.2 Agents à logique déterministe

Agent	**Entrée / Sortie**	**Rôle**
EDA Agent	Dataset → statistiques, corrélations, outliers	Détection d'outliers par règle IQR (1,5×), résumé LLM des observations
Preprocessing Agent	Dataset + décisions EDA → dataset transformé	Imputation, encodage (One-Hot si ≤10 catégories, Label Encoding sinon), décisions calculées dynamiquement par colonne
Evaluation Agent	Modèle entraîné → métriques	Accuracy / Precision / Recall / F1 / ROC AUC ou MAE / MSE / RMSE / R^2, intervalle de confiance à 95% sur le score CV
Meta-Evaluator Agent	État complet → score de confiance global	Audite l'accord SHAP/Permutation, le score du modèle et le nombre d'arbitrages déclenchés
Reporting Agent	État complet → PDF	Compile résultats et arbitrages tracés en rapport PDF (ReportLab)

TABLE 5.1 – Rôle des agents à logique déterministe.

5.3 Model Selection Agent : arbitrage statistique

Le choix du modèle applique un test de Wilcoxon signed-rank sur les scores de validation croisée *par fold*, avec $CV_FOLDS = 10$ (*listing* 5.1) – la justification de cette valeur, initialement fixée à 5, est détaillée section 5.7.

```
CV_FOLDS = 10
RANDOM_STATE = 42

def _compare_top_two(self, results):
    if len(results) < 2:
        return None, None, results[0] if results else None
    best_result, second_result = results[0], results[1]
    best_folds = best_result.get("fold_scores")
    second_folds = second_result.get("fold_scores")
    if not best_folds or not second_folds or len(best_folds) < 2 or len(
       second_folds) < 2:
        return None, None, best_result

    try:
        _, p_value = wilcoxon(best_folds, second_folds)
        p_value = float(p_value)
    except ValueError:
        # wilcoxon leve une exception si toutes les differences par
        # paire sont nulles (scores identiques sur chaque fold) --
        # c'est PRECISEMENT le cas "aucune difference significative"
        p_value = 1.0

    arbitration_entry = None
    winner = best_result
    if p_value > 0.05:
        # arbitrage LLM sur criteres secondaires uniquement
        # (interpretabilite, cout de calcul) -- jamais sur la
        # performance, deja tranchee par le test statistique
        arbitration_response = self.llm.generate_summary(...)
        recommended_name = self._parse_arbitration_recommendation(
            arbitration_response, best_result["model_name"],
            second_result["model_name"],
        )
        if recommended_name == second_result["model_name"]:
            winner = second_result
        arbitration_entry = {
            "agent_name": self.name,
            "trigger": f"Wilcoxon p={p_value:.3f} (no significant
               difference)",
            "llm_arbitration": arbitration_response,
            "parsed_recommendation": recommended_name,
```

```
        }
    return p_value, arbitration_entry, winner
```

Listing 5.1 – `_compare_top_two()` – app/core/agents/model_selection_agent.py (prompts LLM omis pour la concision).

La confiance associée, $confidence = 1 - p_value$, est calculée uniformément dans `execute()`, que le modèle final résulte du test statistique seul ou d'un arbitrage LLM.

5.4 Explainability Agent : accord SHAP / Permutation Importance

L'accord entre les deux méthodes d'attribution est mesuré par une corrélation de rang de Spearman, avec un seuil de désaccord $AGREEMENT_LOW_THRESHOLD = 0{,}3$ (*listing* 5.2).

```
AGREEMENT_LOW_THRESHOLD = 0.3

def _agreement_score(self, shap_importance, perm_importance):
    common_features = [f for f in shap_importance if f in perm_importance]
    if len(common_features) < 2:
        return None
    shap_values = [shap_importance[f] for f in common_features]
    perm_values = [perm_importance[f] for f in common_features]
    try:
        correlation, _ = spearmanr(shap_values, perm_values)
    except Exception:
        return None
    if correlation is None or np.isnan(correlation):
        return None
    return float(correlation)

# --- extrait de execute() ---
agreement_score = self._agreement_score(shap_importance,
    permutation_importance)
if agreement_score is not None and agreement_score <
    AGREEMENT_LOW_THRESHOLD:
    # le LLM explique le desaccord, ne tranche jamais en faveur
    # d'une methode
    arbitration_response = self.llm.generate_summary(...)
    arbitration_log.append({
        "agent_name": self.name,
        "trigger": f"SHAP/permutation␣agreement␣score={agreement_score:.3f}
            "
                   f"␣(<␣{AGREEMENT_LOW_THRESHOLD})",
        "llm_arbitration": arbitration_response,
```

```
28      })
29
30  if agreement_score is not None:
31      confidence = float(max(0.0, agreement_score))
```

Listing 5.2 – `_agreement_score()` et déclenchement de l'arbitrage – app/core/agents/explainability_agent.py (prompt LLM omis).

Il est notable que la fonction retourne `None` – plutôt qu'un chiffre fabriqué – lorsque trop peu de *features* communes existent pour corréler significativement : le système préfère l'absence de confiance à une confiance non fondée.

5.5 Data Understanding Agent et Hyperparameter Optimization Agent

La détection de la colonne cible applique une hiérarchie à quatre niveaux (cible utilisateur explicite, confidence 1,0 ; nom reconnu via l'ensemble `COMMON_TARGET_NAMES`, confidence 0,5–0,9 ; suggestion LLM validée par le ratio de cardinalité, confidence 0,5–0,75 ; repli sur la dernière colonne). Le point clé, vérifié dans le code : l'arbitrage LLM se déclenche dès que cette dernière branche est atteinte – inconditionnellement, sans condition supplémentaire sur un score de confiance.

L'optimisation des hyperparamètres (Optuna, TPE) utilise $N_TRIALS = 20$ et un budget de $TIMEOUT_SECONDS = 60$ pour la première passe, avec validation croisée à $CV_FOLDS_DURING_SEARCH = 3$ plis pendant la recherche. Un retry unique est déclenché si l'amélioration reste sous $MIN_IMPROVEMENT_THRESHOLD = 0{,}01$ (1 %), avec un espace de recherche élargi de $SEARCH_SPACE_WIDEN_MULTIPLIER = 1{,}3$ (+30 %) et un budget total partagé de $TOTAL_TIMEOUT_SECONDS = 90$ secondes entre les deux passes – le temps restant étant simplement calculé par soustraction du temps déjà écoulé.

5.6 Traçabilité : `AgentDecision` et `EMADSState`

Chaque décision est encapsulée dans une structure commune (*listing* 5.3), et les champs de journalisation de `EMADSState` sont fusionnés automatiquement entre agents via `operator.add` plutôt qu'écrasés – garantissant qu'aucun arbitrage n'est perdu au fil du pipeline.

```
1  @dataclass
2  class AgentDecision:
3      agent_name: str
4      decision: str
```

```
    reasoning: str
    confidence: Optional[float] = None
    timestamp: str = field(default_factory=lambda: datetime.now().isoformat
        ())

class EMADSState(TypedDict, total=False):
    # ... (champs de donnees et de resultats omis, cf. Annexe C)

    # Champs "accumulate" -- fusionnes automatiquement par LangGraph
    # via operator.add, jamais ecrases par un agent
    agent_decisions: Annotated[List[AgentDecision], operator.add]
    logs: Annotated[List[str], operator.add]
    errors: Annotated[List[str], operator.add]
    llm_arbitration_log: Annotated[List[Dict[str, Any]], operator.add]
    meta_evaluation: Optional[Dict[str, Any]]
```

Listing 5.3 – `AgentDecision` et champs de traçabilité de `EMADSState` (extrait) – app/core/state/emads_state.py.

5.7 Justification du choix de CV_FOLDS = 10

Le nombre de plis de validation croisée n'est pas un paramètre anodin pour un test de Wilcoxon signed-rank : avec seulement n observations appariées, ce test exact possède un plancher structurel de p-value, $p_{min}(n) = 2 \times (1/2)^n$, en deçà duquel aucune valeur de p ne peut descendre, quelle que soit l'ampleur réelle de l'écart observé. La figure 5.1 trace cette borne.

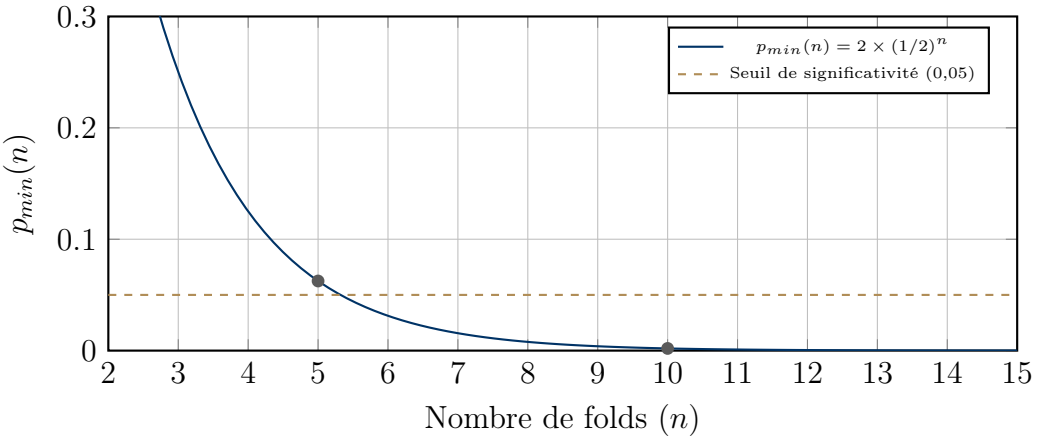


FIGURE 5.1 – Plancher de p-value du test de Wilcoxon signed-rank selon le nombre de folds. À $n = 5$, $p_{min} = 0{,}0625$ reste au-dessus du seuil de 0,05 : le test ne peut structurellement jamais conclure. À $n = 10$, $p_{min} = 0{,}00195$, largement en dessous.

Avec $n = 5$ (valeur initiale du projet), p_{min} dépasse donc toujours le seuil de significativité, quel que soit l'écart de performance réel entre deux modèles – une limite purement

mathématique, indépendante des données. Le passage à $CV_FOLDS = 10$ corrige structurellement ce défaut. La preuve empirique de cette limite sur données réelles, avant et après correction, est présentée au Chapitre 6.

5.8 Défis techniques d'implémentation

— **Classes non 0-indexées** : XGBoost échoue systématiquement lorsque les classes cibles ne sont pas indexées à partir de 0. Un wrapper `LabelEncoder` normalise les classes avant l'entraînement.

— **Détection du séparateur CSV** : un fichier réexporté depuis Excel avec un séparateur `;` n'était initialement pas détecté, le dataset étant lu comme une colonne unique sans exception levée. La détection du séparateur a été rendue explicite en amont du chargement.

L'inventaire complet des neuf anomalies détectées au fil des tests est présenté au Chapitre 6.

Chapitre 6

Validation et résultats

6.1 Méthodologie de validation

La validation d'EMADS a délibérément privilégié des jeux de données construits pour être difficiles plutôt que des cas nominaux, chacun ciblant un mécanisme précis du système (tableau 6.1).

Dataset	**Dimensions**	**Problème**	**Mécanisme éprouvé**
messy_loans.csv	615×9	Classif. binaire	Colonne ID haute cardinalité, colonne constante, 64 % valeurs manquantes, doublons – Preprocessing Agent
fraud_extreme.csv	2500×10	Classif. binaire	Déséquilibre extrême 97 %/3 % – validation croisée stratifiée, boucle bornée du Hyperparameter Optimization Agent
ambiguous_survey_v2.csv	45×5	Classif. multiclasse (5)	Cible sans nom reconnu, petit échantillon – arbitrage inconditionnel du Data Understanding Agent
Régression synthétique	500×12	Régression	A révélé la limite structurelle du test de Wilcoxon (section 6.3)

TABLE 6.1 – Les quatre jeux de données de validation et le mécanisme spécifique éprouvé par chacun.

Cette approche par cas limites a permis de mettre au jour neuf anomalies réelles, présentées section 6.2, dont plusieurs affectent directement l'intégrité des scores de confiance – c'est-à-dire le cœur même de l'explicabilité statistique du système.

6.2 Neuf anomalies détectées et corrigées

Le tableau 6.2 recense les neuf anomalies identifiées au fil des tests. Cinq d'entre elles (#1, #2, #4, #5, #6) affectent directement la fiabilité des scores de confiance ou des textes explicatifs produits par le système – elles constituent, à ce titre, le résultat de validation le plus significatif de ce travail.

#	**Anomalie**	**Correction**
1	Confidence du Model Selection incohérente avec le p-value affiché ; texte "achieved best accuracy" trompeur pour un modèle n'ayant pas le meilleur score	Texte de reasoning recalculé pour refléter fidèlement la décision réellement prise
2	Colonne cible incluse à tort dans la détection d'outliers de l'EDA Agent (règle IQR appliquée à une cible binaire déséquilibrée = faux positif)	Colonne cible exclue de la détection d'outliers
3	Séparateur CSV mal détecté (fichier réexporté Excel en `;`), aucune exception levée, dataset lu comme une seule colonne	Détection explicite du séparateur avant chargement
4	Confidence de l'Explainability Agent gonflée artificiellement (formule `(agreement_score+1)/2` au lieu du score brut)	Confidence recalculée en `max(0, agreement_score)` (cf. *listing* 5.2)
5	Texte de reasoning trompeur après un *winner-switch* LLM en Model Selection (prétendait le meilleur score alors que le *runner-up* avait été retenu)	Reasoning aligné sur le modèle réellement sélectionné après arbitrage
6	Seuil d'arbitrage de la cible rendu inopérant par une formule de confidence graduée introduite dans un refactor antérieur (effet de bord non anticipé)	Déclenchement de l'arbitrage rendu inconditionnel (cf. *listing* 5.1)
7	XGBoost échouait systématiquement sur des classes non 0-indexées (ex. notations 1–5 au lieu de 0–4)	Wrapper `LabelEncoder` ajouté avant l'entraînement

#	**Anomalie**	**Correction**
8	Plancher de p-value structurel du test de Wilcoxon signed-rank à $n = 5$ folds (cf. section 6.3)	Passage à $CV_FOLDS = 10$
9	Meta-Evaluator Agent placé après le Reporting Agent dans le pipeline : son verdict n'apparaissait jamais dans le PDF généré	Réordonnancement du pipeline (Meta-Evaluator avant Reporting)

TABLE 6.2 – Les neuf anomalies détectées et corrigées durant la validation.

6.3 Preuve empirique du plancher de p-value de Wilcoxon

La limite mathématique établie section 5.7 ($p_{min}(5) = 0{,}0625$) s'est vérifiée concrètement sur le dataset de régression synthétique, entre une régression linéaire et un LightGBM aux scores nettement différents (tableau 6.3).

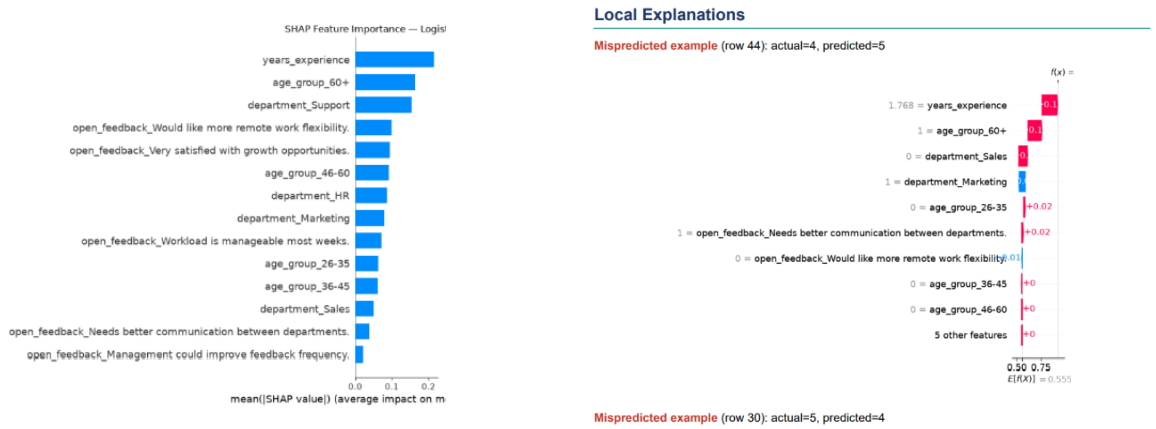
Configuration	**Scores (neg. RMSE)**	p**-value**	**Arbitrage**
$CV_FOLDS = 5$ (avant)	-19080 vs -33086	0,0625	Déclenché à tort
$CV_FOLDS = 10$ (après)	-19080 vs -33086	0,002	Non déclenché

TABLE 6.3 – Même écart de performance, même dataset : le passage de 5 à 10 folds change la conclusion statistique du test.

Avec 5 folds, un écart pourtant important entre les deux modèles ne pouvait jamais être déclaré statistiquement significatif, déclenchant systématiquement un arbitrage LLM superflu. Avec 10 folds, le même écart est immédiatement reconnu comme significatif. Cette correction constitue, au-delà de l'implémentation logicielle, un apport méthodologique propre à ce travail.

6.4 Résultats d'explicabilité et d'évaluation

Les figures 6.1 présentent des extraits directement issus d'un rapport PDF généré par EMADS, illustrant concrètement le fonctionnement du mécanisme d'explicabilité et les métriques produites par l'Explainability Agent.



(a) SHAP Feature Importance. (b) Local Explanations.

FIGURE 6.1 – Extraits d'explicabilité (SHAP) pour le modèle sélectionné par EMADS sur `ambiguous survey v2.csv`.

6.5 Traçabilité en pratique

La figure 6.2 montre la section *Statistical Verdict / LLM Interpretation* d'un rapport PDF généré par EMADS – illustration concrète du journal `llm_arbitration_log` présenté section 5, après correction du texte de reasoning trompeur identifié comme anomalie #1/#5 (tableau 6.2).

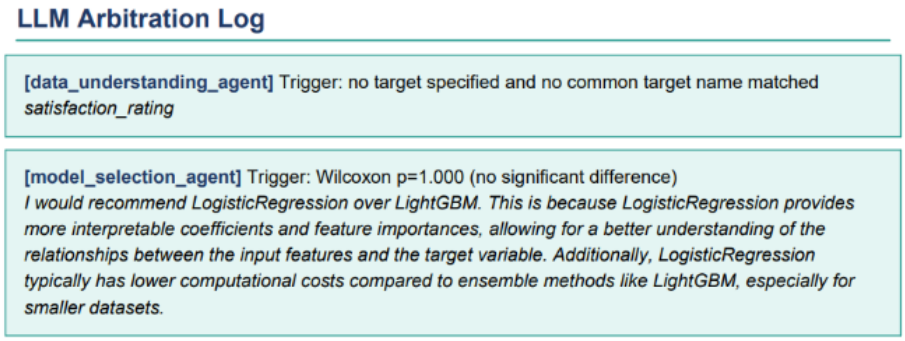
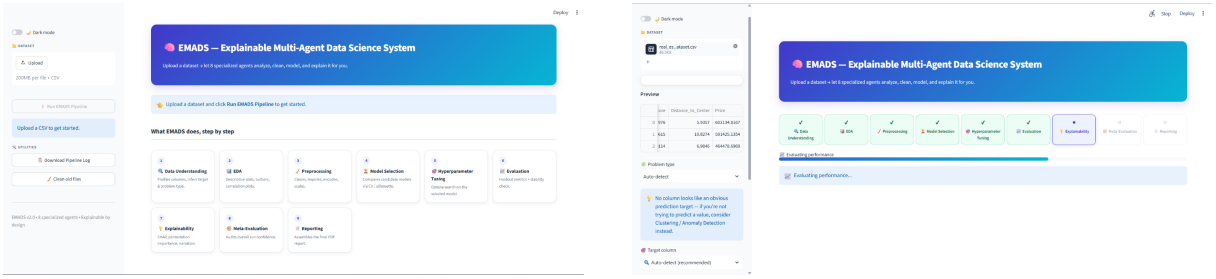


FIGURE 6.2 – Section "Statistical Verdict / LLM Interpretation" d'un rapport PDF EMADS.

6.6 Aperçu de l'interface utilisateur

L'interface Streamlit expose ces résultats de façon interactive (figure 6.3) ; les captures d'écran complémentaires sont regroupées en Annexe. Le prototype est accessible en ligne à l'adresse suivante : `https://emads-explain.streamlit.app/`.



(a) Onglet Overview (badge Meta-Evaluator). (b) Stepper de progression du pipeline.

FIGURE 6.3 – Aperçu de l'interface utilisateur Streamlit d'EMADS.

6.7 Synthèse

Les résultats de ce chapitre valident moins la performance des modèles produits par EMADS que la fiabilité de son infrastructure d'explicabilité elle-même : cinq des neuf anomalies corrigées concernaient directement l'intégrité des scores de confiance ou des explications produites, et la correction du plancher de p-value du test de Wilcoxon a rendu le mécanisme d'arbitrage de Model Selection statistiquement valide plutôt que systématiquement déclenché par construction. C'est cette rigueur méthodologique, plus que la seule exactitude des modèles entraînés, qui répond directement à la question de recherche posée en introduction de ce rapport.

Chapitre 7

Discussion, limites et perspectives

7.1 Une question de conception au-delà du cahier des charges

La mission confiée par Madame Abir Bala (cf. Chapitre 1) portait sur la conception d'un agent capable d'automatiser un projet de Data Science tout en rendant ses décisions explicables. Pour traduire cette exigence en choix d'architecture concret, il m'a semblé utile, en cours de conception, de m'appuyer sur la distinction théorique entre workflow et agent posée au Chapitre 2 et développée au Chapitre 4. Ce cadrage n'était pas une consigne explicite mais une initiative personnelle de structuration, adoptée pour donner une base théorique solide aux choix d'architecture présentés dans ce rapport – je le précise ici par souci de transparence sur ma propre démarche.

Cette question invite à une réflexion plus large sur les compromis de conception envisageables pour ce type de système. Un système pleinement agentique, où un LLM orchestrerait librement l'enchaînement des étapes du pipeline, présenterait un avantage réel : une capacité d'adaptation à des structures de données ou des cas d'usage imprévus, que la séquence fixe d'EMADS ne peut pas offrir. Le choix fait ici – un workflow déterministe devenant agentique seulement à trois points précis – privilégie délibérément la reproductibilité et la vérifiabilité statistique sur cette flexibilité. Ce compromis n'est pas présenté comme universellement supérieur : il est justifié pour un système dont l'objectif premier est la fiabilité des explications produites, mais il aurait pu être posé différemment si l'objectif du stage avait été, par exemple, la capacité d'EMADS à s'adapter à des types de problèmes de Data Science non anticipés. Cette tension entre adaptabilité et vérifiabilité reste, à l'issue de ce stage, une question ouverte plutôt que tranchée – elle est reprise en perspective section 7.4.

7.2 Réponses aux remarques formulées en réunion de suivi

Deux remarques soulevées lors du point d'étape avec l'encadrante (cf. Chapitre 1) trouvent ici une réponse explicite.

Comment un simple appel API LLM peut-il "choisir le meilleur modèle" ? La réponse tient dans le mécanisme même détaillé section 4.5 : le LLM n'arbitre jamais un cas de différence de performance réelle – celle-ci est déjà tranchée par le test de Wilcoxon signed-rank. Il n'intervient que dans la zone grise où la statistique elle-même déclare ne pas savoir trancher ($p > 0{,}05$), et selon une règle méthodologique reconnue (*one standard error*, Breiman et al., 1984) plutôt qu'une opinion improvisée. Le LLM ne calcule donc jamais un score : il départage deux modèles déjà jugés statistiquement équivalents.

Téléchargement du modèle entraîné. Cette fonctionnalité, demandée lors du point d'étape, était déjà implémentée au moment de la remarque – le modèle entraîné (`.pkl`) est accessible depuis l'interface Streamlit et référencé dans `EMADSState` (champ `model_path`, cf. *listing* 5.3).

7.3 Limites

Trois limites méritent d'être assumées explicitement, plutôt que minimisées.

Le module de clustering et de détection d'anomalies, présent dans le code mais encore en cours de validation, ne bénéficie pas du même niveau de rigueur statistique que le reste du système : SHAP y est correctement désactivé (l'attribution de features n'a pas de sens direct pour ces tâches non supervisées), mais son score de confiance repose sur une heuristique fondée sur l'écart de silhouette entre clusters, et non sur un test statistique comparable à ceux développés pour les tâches supervisées. Cette hétérogénéité de rigueur au sein du même système est une limite réelle, non un détail d'implémentation secondaire : elle signifie que l'explicabilité d'EMADS, telle qu'elle a été validée dans ce rapport, ne s'étend pas encore uniformément à l'ensemble des cas d'usage que le système prétend couvrir.

Le périmètre de validation, bien que délibérément construit autour de cas limites plutôt que de cas nominaux (Chapitre 6), reste circonscrit à quatre jeux de données. Cette diversité contrôlée a permis de révéler neuf anomalies réelles, mais ne constitue pas une preuve d'absence d'anomalies supplémentaires sur des structures de données non couvertes par ces quatre cas.

La dépendance à un unique fournisseur LLM (Groq, `llama-3.1-8b-instant`) pour les trois mécanismes d'arbitrage introduit une dépendance externe dont la disponibilité et la latence ne sont pas contrôlées par EMADS – un point pertinent dans la mesure où

ces arbitrages, bien que statistiquement encadrés, restent une étape nécessaire du pipeline lorsqu'ils se déclenchent.

7.4 Perspectives d'évolution

Quatre axes d'évolution ont été identifiés à l'issue de ce travail :

— **Évaluation empirique multi-datasets systématique** : étendre la validation au-delà des quatre jeux de données actuels, vers un protocole couvrant une plus large diversité de structures de données réelles.

— **Mémoire persistante des runs** (SQLite) : conserver l'historique des décisions et arbitrages d'un run à l'autre, pour analyser la stabilité des choix du système dans le temps.

— **Comparaison expérimentale workflow vs pleinement agentique** : mettre à l'épreuve, empiriquement plutôt que théoriquement, la tension discutée section 4.5 entre flexibilité d'un système pleinement agentique et vérifiabilité statistique d'un workflow à arbitrage conditionnel.

— **Stabilité temporelle des explications SHAP** : évaluer si les explications produites par le système restent cohérentes d'une exécution à l'autre sur un même dataset, en écho au risque d'instabilité des méthodes locales signalé par Alvarez-Melis et Jaakkola (2018) (cf. Chapitre 2).

Conclusion générale

Ce rapport est parti d'une question posée en introduction : comment un système multi-agents peut-il automatiser un pipeline de science des données tout en rendant ses décisions explicables et dignes de confiance, plutôt que de se contenter d'une justification en langage naturel produite après coup ? EMADS y répond en fondant chacune de ses décisions critiques – sélection de modèle, détection de la colonne cible, accord entre méthodes d'explicabilité – sur un test statistique explicite, n'ayant recours à un LLM que dans la zone grise où ce test se déclare lui-même inconclusif. Cette exigence, formulée par Madame Abir Bala au début du stage, a structuré l'ensemble de la démarche présentée dans ce rapport.

L'apport principal de ce travail n'est donc pas la seule automatisation d'un pipeline de *Machine Learning* – un problème aujourd'hui largement couvert par l'AutoML – mais la démonstration qu'une explicabilité vérifiable est possible à condition de la fonder sur des méthodes statistiques rigoureuses plutôt que sur une génération de texte plausible. L'architecture hybride workflow/agentique (Chapitre 4) en est le moyen, non une fin en soi : la validation menée au Chapitre 6 en a directement bénéficié, avec cinq des neuf anomalies corrigées touchant l'intégrité même des scores de confiance, et la découverte du plancher structurel de p-value du test de Wilcoxon signed-rank à cinq *folds*, corrigée en passant à dix – un apport méthodologique propre à ce travail, qu'aucune métrique de performance globale n'aurait révélé.

Sur le plan personnel, ce stage a permis de mener un travail de recherche appliquée avec un niveau de rigueur académique, tout en restant ancré dans un contexte industriel concret chez Harmony Technology – l'occasion, surtout, d'apprendre à distinguer une explication convaincante d'une explication prouvée. Les perspectives identifiées au Chapitre 7, en particulier une comparaison expérimentale entre l'approche hybride retenue ici et une architecture pleinement agentique, ouvrent la voie à la poursuite de ce travail au-delà de ce stage, avec l'ambition, formulée dès la mission initiale, d'une contribution scientifique suffisamment solide pour envisager une valorisation par publication.

Bibliographie

[1] Ali, S., Abuhmed, T., El-Sappagh, S., et al. (2023). *Explainable Artificial Intelligence (XAI) : What we know and what is left to attain Trustworthy Artificial Intelligence*. Information Fusion, 99, 101805.

[2] Alvarez-Melis, D., & Jaakkola, T. S. (2018). *On the Robustness of Interpretability Methods*. Présenté au 2018 ICML Workshop on Human Interpretability in Machine Learning.

[3] Anthropic (2024). *Building Effective Agents*. `https://www.anthropic.com/research/building-effective-agents`.

[4] Breiman, L., Friedman, J. H., Olshen, R. A., & Stone, C. J. (1984). *Classification and Regression Trees*. Wadsworth International Group.

[5] Dietterich, T. G. (1998). *Approximate Statistical Tests for Comparing Supervised Classification Learning Algorithms*. Neural Computation, 10(7), 1895–1923.

[6] Lundberg, S. M., & Lee, S.-I. (2017). *A Unified Approach to Interpreting Model Predictions*. Advances in Neural Information Processing Systems (NeurIPS), 30.

[7] Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). *"Why Should I Trust You ?" : Explaining the Predictions of Any Classifier*. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 1135–1144.

[8] Russell, S., & Norvig, P. (2020). *Artificial Intelligence : A Modern Approach* (4e éd.). Pearson.

[9] Shapley, L. S. (1953). *A Value for n-Person Games*. In Contributions to the Theory of Games, Volume II (Annals of Mathematics Studies, No. 28). Princeton University Press, 307–317.

Annexe A

Captures d'écran complémentaires de l'interface

Cette annexe regroupe des captures d'écran complémentaires de l'interface Streamlit, en complément des figures 4.2 et 6.3 présentées respectivement aux Chapitres 4 et 6.

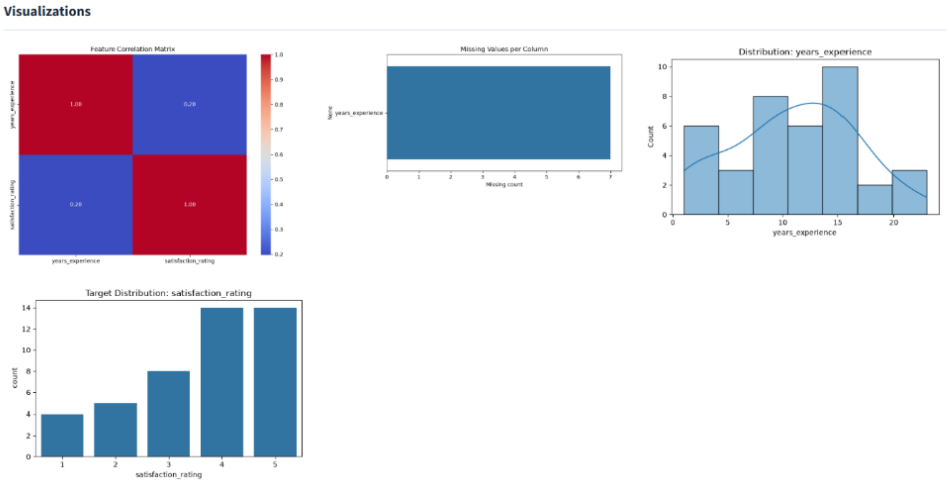


FIGURE A.1 – Vue du rapport d'analyse exploratoire (EDA) dans l'interface.

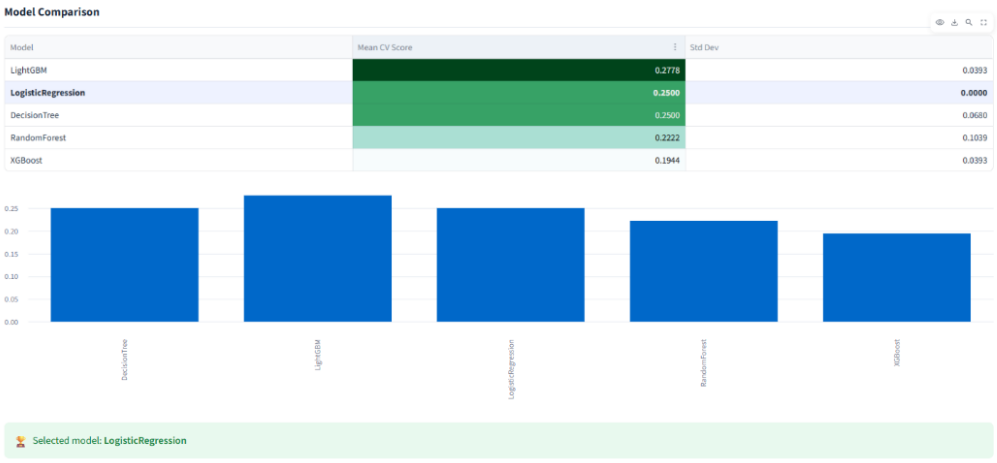


FIGURE A.2 – Tableau de comparaison des modèles candidats (Model Selection Agent).

Annexe B

Exemple de journal de traçabilité

Le journal `llm_arbitration_log`, présenté au Chapitre 5, enregistre chaque arbitrage déclenché avec son agent d'origine, son déclencheur statistique exact et la réponse du LLM. La structure ci-dessous illustre le format d'une entrée type ;

```
{
    "agent_name": "ModelSelectionAgent",
    "trigger": "Wilcoxon p=<valeur> (no significant difference)",
    "llm_arbitration": "<reponse textuelle du LLM>",
    "parsed_recommendation": "<nom du modele retenu>"
}
```

Listing B.1 – Structure d'une entrée du journal d'arbitrage (schéma).

Annexe C

Définition complète de `EMADSState`

Par souci de concision, le *listing* 5.3 du Chapitre 5 n'a présenté que les champs de traçabilité de `EMADSState`. La définition complète, telle qu'implémentée dans `app/core/state/emads_s` est reproduite ci-dessous.

```python
SUPERVISED_PROBLEM_TYPES = {"classification", "regression"}
UNSUPERVISED_PROBLEM_TYPES = {"clustering", "anomaly_detection"}

class EMADSState(TypedDict, total=False):
    session_id: str
    created_at: str
    current_step: str

    dataset_path: str
    dataset_name: Optional[str]
    target_column: Optional[str]
    problem_type: Optional[str]

    schema_info: Optional[Dict[str, Any]]

    eda_summary: Optional[str]
    eda_stats: Optional[Dict[str, Any]]
    generated_plots: Optional[List[str]]

    preprocessing_report: Optional[Dict[str, Any]]
    preprocessed_data_path: Optional[str]
    selected_features: Optional[List[str]]

    candidate_models_results: Optional[List[Dict[str, Any]]]
    selected_model_name: Optional[str]
    model_path: Optional[str]
    model_selection_summary: Optional[str]

    best_hyperparameters: Optional[Dict[str, Any]]
    optimization_summary: Optional[Dict[str, Any]]

    metrics: Optional[Dict[str, Any]]
    evaluation_plots: Optional[List[str]]
```

```
34
35      permutation_importance: Optional[Dict[str, float]]
36      feature_importance: Optional[Dict[str, float]]
37      shap_plots: Optional[List[str]]
38      explainability_summary: Optional[str]
39
40      local_explanations: Optional[List[Dict[str, Any]]]
41      explanation_agreement_score: Optional[float]
42
43      report_path: Optional[str]
44
45      # Champs "accumulate" -- fusionnes automatiquement par LangGraph
46      # via operator.add, jamais ecrases par un agent
47      agent_decisions: Annotated[List[AgentDecision], operator.add]
48      logs: Annotated[List[str], operator.add]
49      errors: Annotated[List[str], operator.add]
50      llm_arbitration_log: Annotated[List[Dict[str, Any]], operator.add]
51
52      meta_evaluation: Optional[Dict[str, Any]]
```

Listing C.1 – Définition complète de `EMADSState` – app/core/state/emads_state.py.